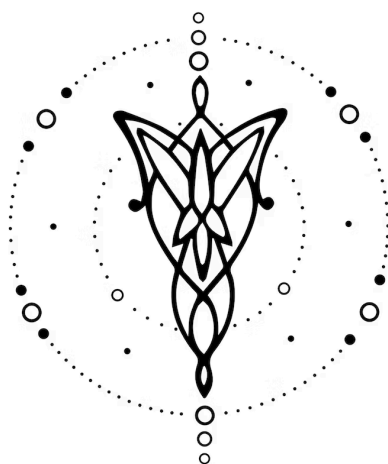




Especificación

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	1
3. Dominio	2
4. Construcciones	2
5. Casos de prueba	4
6. Ejemplos	7

1. Equipo

Apellido	Legajo	E-mail
Yelmini	64133	cyelmini@itba.edu.ar
Gonzalez Porzio	64028	mgonzalezporzio@itba.edu.ar
Ravera	65690	jravera@itba.edu.ar
Grelle	65221	agrelle@itba.edu.ar

2. Repositorio

La solución será versionada en: <https://github.com/azugrelle/TPE-TLA>.

3. Dominio

Desarrollar un lenguaje imperativo basado en instrucciones que permita definir una hora, manipularla mediante operaciones aritméticas y funciones, personalizar su representación visual y generar como salida un documento HTML con la representación gráfica de un reloj analógico.

El lenguaje opera sobre un estado interno que representa un reloj con una hora actual y un conjunto de propiedades visuales. Cada instrucción modifica dicho estado de forma secuencial: se inicializa el reloj con una hora en formato HH:MM, se aplican operaciones de suma y resta de tiempo, funciones de ajuste como redondeo o reemplazo de componentes, modificadores de estilo para personalizar colores y numeración, y opcionalmente se realiza una conversión de zona horaria. Finalmente, la instrucción render genera el documento de salida con el reloj resultante.

La implementación satisfactoria de este lenguaje permitiría generar representaciones visuales de horas calculadas de forma rápida y personalizable, útiles para documentación, material educativo o herramientas de planificación horaria.

4. Construcciones

El lenguaje desarrollado debería ofrecer las siguientes construcciones, prestaciones y funcionalidades:

Inicialización

(I). Se podrá inicializar el reloj mediante la instrucción clock HH:MM.

Sintaxis: clock <HH:MM>

Render

(II). Se podrá generar el documento de salida mediante la instrucción render.

Sintaxis: render

Operaciones aritméticas

(I). Se podrá incrementar un valor temporal mediante el operador add.

Sintaxis: add <ENTERO POSITIVO> <UNIDAD>

(II). Se podrá decrementar un valor temporal mediante el operador sub.

Sintaxis: sub <ENTERO POSITIVO> <UNIDAD>

(III). Las unidades de tiempo soportadas son:

- hours

- minutes

(IV). Se podrán encadenar múltiples operaciones aritméticas de forma secuencial, aplicándose en el orden en que aparecen.

(V). Las operaciones deben manejar correctamente los desbordes entre unidades de tiempo. En particular:

- Sumar minutos puede incrementar horas
- Restar minutos puede decrementar horas.

(VI). El formato del tiempo será de 24 horas.

Funciones

(I). Se podrá reemplazar el componente hora del reloj mediante la instrucción `set hour` seguida de un entero positivo. El valor debe pertenecer al rango 0-23. Los minutos se conservan sin cambios.

Sintaxis: `set hour <ENTERO POSITIVO>`

(II). Se podrá reemplazar el componente minutos del reloj mediante la instrucción `set minute` seguida de un entero positivo. El valor debe pertenecer al rango 0-59. La hora se conserva sin cambios.

Sintaxis: `set minute <ENTERO POSITIVO>`

(III). Se podrá redondear la hora mostrada al múltiplo de 5 minutos más cercano mediante la instrucción `round to 5 minutes`. Si el redondeo excede el minuto 59, la hora se incrementa respetando aritmética modular sobre 24 horas.

Sintaxis: `round to 5 minutes`

(IV). Se podrá avanzar el reloj hasta el inicio de la siguiente hora exacta mediante la instrucción `next hour`. Esta instrucción fija minutos en 00 y no equivale a sumar una hora. Si el reloj muestra 23:XX, el resultado es 00:00.

Sintaxis: `next hour`

Estilo

Se podrá personalizar el estilo del reloj mediante instrucciones que modifican el estado visual antes de ejecutar render.

(I). Se podrá definir el color de las agujas mediante la instrucción `color` seguida de un color.

Sintaxis: `color <COLOR>`

(II). Se podrá definir el color de fondo del reloj mediante la instrucción `background` seguida de un color.

Sintaxis: background <COLOR>

(III). Se podrá definir el color del borde del reloj mediante la instrucción border seguida de un color.

Sintaxis: border <COLOR>

(IV). Se podrá definir el tipo de numeración de las marcas horarias mediante la instrucción numbers seguida del tipo. Los tipos aceptados son arabic y roman.

Sintaxis: numbers <TIPO>

(V). Los colores aceptados son: black, white, green, red, blue.

(VI). Si no se especifica ninguna instrucción de estilo, el reloj se generará con un estilo por defecto: agujas negras, fondo blanco, números arábigos y borde negro.

(VII). Las instrucciones de estilo se pueden indicar en cualquier orden antes del render.

Vista y cambio de zona horaria

(I). Se podrá obtener la zona horaria del reloj mediante expresiones del tipo UTC o UTC±H.

Sintaxis: UTC<±H>

(II). La expresión UTC representa el tiempo universal coordinado, siendo equivalente a UTC+0.

(III). Los símbolos + o - indican el desplazamiento respecto a UTC.

(IV). El valor H representa la cantidad de horas de desplazamiento y debe ser un entero.

(V). El valor H debe pertenecer a un rango válido de zonas horarias (por ejemplo, $-12 \leq H \leq +14$).

(VI). Se podrá realizar la conversión entre zonas horarias mediante el operador ->.

Sintaxis: <ZonaOrigen> -> <ZonaDestino>

Si no se especifica previamente una hora mediante clock, se tomará la hora actual del sistema interpretada en la <ZonaOrigen> como punto de partida para la conversión. En el ejemplo 2 en la sección de ejemplos se especifica previamente una hora mediante clock.

(VII). <ZonaOrigen> y <ZonaDestino> deberán ser zonas horarias válidas en formato UTC o UTC±H.

(VIII). Toda conversión deberá involucrar exactamente una zona de origen y una zona de destino.

Bucle repeat

(I) Se podrá repetir un bloque de instrucciones mediante la palabra repeat.

Sintaxis:

```
repeat <ENTERO> {
    <instrucciones>
}
```

(II). El valor de repeticiones debe ser un entero positivo mayor a 0. Valores iguales a 0 o negativos serán rechazados

Condicionales

(I). Se podrá ejecutar un bloque de instrucciones de forma condicional mediante la instrucción if. La condición se evalúa sobre el estado actual del reloj y, si resulta verdadera, se ejecutan las instrucciones contenidas en el bloque.

Sintaxis:

```
if (<condición>) {
    <instrucciones>
}
```

(II). Opcionalmente, se podrá definir un bloque alternativo mediante la instrucción else, que se ejecutará únicamente cuando la condición del if resulte falsa.

Sintaxis:

```
if (<condición>) {
    <instrucciones>
} else {
    <instrucciones>
}
```

(III). Las condiciones se podrán formular sobre las componentes hour y minute del reloj, comparándolas con un valor entero. Los comparadores soportados son: ==, !=, <=, >=, < y >.

Sintaxis: <componente> <comparador> <ENTERO>

Operadores lógicos

Las condiciones pueden combinarse utilizando operadores lógicos para formar expresiones más complejas:

- <condición> AND <condición>
- <condición> OR <condición>
- NOT <condición>

También se permite el uso de paréntesis para agrupar condiciones: (<condición>)

Los operadores lógicos respetan el siguiente orden de precedencia:

1. NOT
2. AND
3. OR

Esto implica, por ejemplo, que “A OR B AND C” se interpreta como “A OR (B AND C)”

5. Casos de prueba

Se proponen los siguientes casos iniciales de **prueba de aceptación**:

Operaciones aritméticas

- (I). Un programa que permita realizar suma de horas.
- (II). Un programa que permita realizar suma de minutos.
- (III). Un programa que permita realizar resta de horas.
- (IV). Un programa que permita realizar resta de minutos.
- (V). Un programa que combine múltiples operaciones aritméticas utilizando unidades válidas.
- (VI). Un programa que permita realizar suma de minutos con desborde positivo hacia las horas.

Funciones

- (I). Un programa que reemplace la componente hora del reloj conservando minutos.
- (II). Un programa que reemplace la componente minutos del reloj conservando hora.
- (III). Un programa que aplique set hour con el valor límite 0.
- (IV). Un programa que aplique set hour con el valor límite 23.
- (V). Un programa que aplique set minute con el valor límite 0.
- (VI). Un programa que aplique set minute con el valor límite 59.
- (VII). Un programa que redondee la hora al múltiplo de 5 minutos más cercano hacia abajo.
- (VIII). Un programa que redondee la hora al múltiplo de 5 minutos más cercano cruzando la hora.
- (IX). Un programa que avance el reloj a la siguiente hora exacta.

Estilo

- (I). Un programa que genere un reloj con agujas azules y fondo negro.
- (II). Un programa que genere un reloj con números romanos.
- (III). Un programa que combine todos los modificadores de estilo.
- (IV). Un programa que combine operación aritmética con estilo.
- (V). Un programa que genere un reloj con estilo por defecto.

Cambio de zona horaria

- (I). Un programa que convierta una hora desde UTC-3 a UTC+0.
- (II). Un programa que establezca la zona horaria a UTC-3.

Bucle repeat

- (I). Un programa que repita una operación aritmética un número fijo de veces.
- (II). Un programa que repita múltiples instrucciones dentro de un mismo bloque repeat.
- (III). Un programa que utilice repeat con valor 1.

Condicionales

- (I). Un programa que ejecute un bloque de instrucciones cuando la hora sea igual a un número entero positivo dado.
- (II). Un programa que ejecute un bloque de instrucciones cuando los minutos sean igual a un número entero positivo dado.
- (III). Un programa que ejecute un bloque de instrucciones cuando la hora sea distinta a un número entero positivo dado.
- (IV). Un programa que ejecute un bloque de instrucciones cuando los minutos sean distintos a un número entero positivo dado.
- (V). Un programa que ejecute un bloque de instrucciones cuando la hora sea mayor que un número entero positivo dado.

(VI). Un programa que ejecute un bloque de instrucciones cuando los minutos sean mayores que un número entero positivo dado.

(VII). Un programa que ejecute un bloque de instrucciones cuando la hora sea menor que un número entero positivo dado.

(VIII). Un programa que ejecute un bloque de instrucciones cuando los minutos sean mayores que un número entero positivo dado.

(IX). Un programa que ejecute un bloque de instrucciones cuando la hora sea mayor o igual que un número entero positivo dado.

(X). Un programa que ejecute un bloque de instrucciones cuando los minutos sean mayores o iguales que un número entero positivo dado.

(XI). Un programa que ejecute un bloque de instrucciones cuando la hora sea menor o igual que un número entero positivo dado.

Operadores lógicos

(I). Un programa que ejecute un bloque de instrucciones cuando dos condiciones simples se cumplan simultáneamente utilizando el operador AND.

(II). Un programa que ejecute un bloque de instrucciones cuando al menos una de dos condiciones simples se cumpla utilizando el operador OR.

(III). Un programa que ejecute un bloque de instrucciones cuando una condición simple no se cumpla mediante el operador NOT.

(IV). Un programa que combine condiciones simples mediante los operadores AND y OR respetando la precedencia definida.

(V). Un programa que utilice paréntesis para modificar el orden de evaluación de una condición compuesta.

Se proponen los siguientes casos iniciales de **prueba de rechazo**:

Operaciones aritméticas

(I). Un programa que contenga un operador inválido.

(II). Un programa que utilice una unidad inválida.

(III). Un programa que utilice una cantidad negativa.

(IV). Un programa que utilice una cantidad no entera.

(V). Un programa que omita la unidad en una operación.

(VI). Un programa que omita la cantidad en una operación.

(VII). Un programa que utilice operaciones sin una hora previamente definida.

Funciones

(I). Un programa con la función set hour con una cantidad mayor a 23.

(II). Un programa que utiliza la función set hour con un valor negativo.

(III). Un programa que utiliza la función set hour con un valor no entero.

(IV). Un programa que utiliza la función set minute con un valor fuera del rango válido (mayor a 59).

(V). Un programa que utiliza la función set minute con un valor negativo.

(VI). Un programa que utiliza la función set minute con un valor no entero.

(VII). Un programa que utiliza la instrucción round to 5 minutes con una sintaxis incorrecta.

(VIII). Un programa que utiliza la instrucción round to 5 minutes con una unidad inválida.

(IX). Un programa que utiliza la instrucción next hour con parámetros adicionales.

(X). Un programa que utiliza la instrucción next hour con sintaxis incorrecta.

(XI). Un programa que utiliza cualquiera de las funciones sin haber definido previamente una hora mediante la instrucción clock.

Estilo

(I). Un programa que use un color no definido.

(II). Un programa que use un tipo de numeración inválido.

(III). Un programa que use background sin indicar el color.

Cambio de zona horaria

(I). Un programa que use un offset fuera de rango (UTC+25).

(II). Un programa que use formato incorrecto (UTC+3:5).

(III). Un programa que use una zona mal formada (UTC++3).

Bucle repeat

(I). Un programa que utilice repeat sin indicar la cantidad de repeticiones.

(II). Un programa que utilice repeat con un valor negativo.

(III). Un programa que utilice repeat con un valor no entero.

(IV). Un programa que utilice repeat sin bloque de llaves.

(V). Un programa que utilice repeat sin haber definido previamente una hora mediante clock.

Condicionales

(I). Un programa que utilice un comparativo inválido dentro de una condición (ej: "===", "<>").

(II). Un programa que utilice un identificador distinto de hour o minute dentro de una condición.

(III). Un programa que omita el comparativo en una condición.

(IV). Un programa que omita el valor entero de comparación en una condición.

(V). Un programa que omita la componente temporal sobre la que se realiza la comparación.

(VI). Un programa que utilice un valor no entero dentro de una condición.

(VII). Un programa que utilice un valor negativo dentro de una condición..

(VIII). Un programa que no abra o cierre correctamente los paréntesis del bloque condicional.

Operadores lógicos

(I). Un programa que utilice un operador lógico no definido por el lenguaje (ej: XOR).

(II). Un programa que intente combinar dos condiciones sin un operador lógico entre ellas.

(III). Un programa que utilice el operador AND sin una condición válida a su derecha.

(IV). Un programa que utilice el operador OR sin una condición válida a su derecha.

(V). Un programa que utilice el operador AND sin una condición válida a su izquierda.

(VI). Un programa que utilice el operador OR sin una condición válida a su izquierda.

(VII). Un programa que utilice el operador NOT sin una condición válida a su derecha.

(VIII). Un programa que utilice el operador NOT con una condición a su izquierda y no a su derecha.

6. Ejemplos

```
//ejemplo 1
clock 14:00
add 2 hours
color red
background green
numbers arabic
render
```

Genera un reloj analógico mostrando la hora resultante de sumar 2 horas, con agujas rojas, fondo verde y números árabigos.

```
//ejemplo 2
clock 23:45
UTC-3 -> UTC+0
color blue
background black
numbers roman
render
```

Genera un reloj que convierte las 23:45 de Argentina (UTC-3) a UTC+0 (quedaría 02:45), con agujas azules, fondo negro y números romanos.

```
// ejemplo 3: operaciones combinadas con redondeo
clock 09:32
add 1 hours
sub 7 minutes
round to 5 minutes
border red
color green
render
```

Arranca en 09:32, suma 1 hora → 10:32, resta 7 minutos → 10:25, redondea al múltiplo de 5 más cercano → 10:25 (ya es múltiplo de 5), con borde rojo y agujas verdes.

```
// ejemplo 4: caso límite con next hour
clock 23:30
next hour
color black
background black
border blue
render
```

El programa comienza en 23:30. La instrucción next hour avanza el reloj al inicio de la siguiente hora exacta, lo que implica 00:00 debido al comportamiento modular sobre 24 horas. Finalmente, se renderiza el reloj con agujas y fondo negros, y borde azul.

```
// ejemplo 5: uso de bucle
clock 10:00
repeat 3 {
  add 5 minutes
}
render
```

El programa comienza en 10:00. El bloque repeat 3 ejecuta la instrucción add 5 minutes tres veces consecutivas, sumando un total de 15 minutos. El resultado final es 10:15, que luego se renderiza.

```
// ejemplo 6: condicional simple verdadero
clock 10:45
if (minute >= 30) {
  next hour
} else {
  set hour 15
}
render
```

El programa comienza en 10:45. La condición minute >= 30 es verdadera ($45 \geq 30$), por lo que se ejecuta next hour.

La instrucción next hour avanza el reloj a 11:00. El bloque else no se ejecuta.

```
// ejemplo 7: operador lógico con expresión falsa
clock 10:45
if (minute >= 30 AND hour == 11) {
  next hour
} else {
  set hour 15
}
render
```

El programa comienza en 10:45. La condición compuesta `minute >= 30 AND hour == 11` resulta falsa, ya que aunque `minute >= 30` es verdadera, `hour == 11` no se cumple (la hora es 10).

Por lo tanto, se ejecuta el bloque `else`, que establece la hora en 15, conservando los minutos. El resultado final es 15:45.

```
// ejemplo 8: AND + OR (precedencia de operadores)
clock 10:20

if (hour == 10 OR minute < 30 AND hour == 12) {
    set minute 0
}
render
```

Arranca en 10:20 y la condición resulta verdadera, ya que `hour == 10` se cumple.

Por lo tanto, se ejecuta la instrucción `set minute 0` y el reloj queda en 10:00.

```
// ejemplo 9: uso de paréntesis
clock 10:20

if ((hour == 10 OR minute < 30) AND hour == 12) {
    set minute 0
}
render
```

En este caso, los paréntesis modifican el orden de evaluación.

Arranca en 10:20 y la condición resulta falsa, ya que aunque `hour == 10 OR minute < 30` es verdadera, `hour == 12` no se cumple.

Por lo tanto, no se ejecuta la instrucción y el reloj permanece en 10:20.